MASTER IMPLEMENTATION PLAN

Customizable, Non-AI Personal Productivity OS

Android + Windows • Local-only data • Minimal, glassy, premium UI

Design direction: Nothing-inspired monochrome widgets, Apple-like spacing and transitions, restrained frosted glass, and a configurable dashboard that connects tasks, calendar, media, reminders, routines, voice input and supported device controls.

Core principle: The visual design should feel unified, but every widget must remain useful, fast, accessible and understandable. The app is not a chatbot and does not require AI models, cloud services or a backend.

Prepared as a build-oriented specification with widget inventory, integration map, architecture, data model, notification reliability, Android platform limits, phased roadmap and testing plan.

1. Product vision and non-negotiable requirements

Build a personal operating-system-style productivity application for Android and Windows. Users manually enter information or dictate it with voice input, then organize it through tasks, calendars, reminders, projects, routines, goals, notes and customizable screens. The experience should feel like a calm, premium control center—not a conventional cluttered task manager.

• **Platforms:** Android and Windows desktop, sharing as much Flutter/Dart code as practical.

• **Data:** Local-only by default. No required account, cloud backend, telemetry service or automatic synchronization.

• **AI:** No conversational AI and no AI-dependent core functionality. Use deterministic rules and ordinary operating-system APIs.

• **Input:** Manual entry and voice dictation with an editable confirmation step.

• **Customization:** Users can add, remove, reorder, resize and configure widgets; create multiple screens; choose themes and save layouts.

• **Integrations:** Notifications, Android home-screen widgets, shortcuts, share-to-capture, voice, audio controls and supported system information.

• **Privacy:** Keep data local. Offer explicit export/import and optional encrypted backup.

• **Reliability:** Persist reminder data before scheduling it with the operating system; reconcile scheduled alarms after relevant system changes.

• **Design:** Minimal, monochrome-first, glassy where appropriate, fluid and responsive; do not sacrifice performance for visual effects.

Important scope note

The plan contains 24 user-facing widget types. They are not 24 separate third-party integrations: many are reusable Flutter components powered by a smaller set of shared services and native APIs.

2. UI/UX direction and visual design rules

2.1 Visual references and feel

• **Nothing-inspired:** monochrome palette, bold typography, restrained dot-matrix details, large time/date and clean card layouts.

• **Apple-inspired polish:** generous spacing, clear hierarchy, smooth transitions, consistent corner radii, subtle borders and accessible text.

• **Personal OS:** one configurable dashboard with separate screens for study, work, daily life, routines and media.

• **Glass surfaces:** frosted/translucent cards, selective blur, subtle gradients and layered depth—not blur on every component.

2.2 Design rules

• **Default appearance:** graphite or near-black surfaces, off-white text and one restrained accent color. Support a light theme as well.

• **Typography:** large clock and important metrics; compact metadata; readable task text; a consistent type scale.

• **Motion:** use short, interruptible transitions for navigation, card expansion, task completion and sheets. Avoid constant decorative animation.

• **Customization:** allow layout, theme and widget-size changes without letting users accidentally break visual consistency.

• **Performance:** blur and transparency must degrade gracefully on lower-end devices. The app must remain usable without them.

• **Accessibility:** semantic labels, adequate contrast, scalable text, keyboard support on Windows, touch-friendly controls and reduced-motion options.

• **Useful density:** show the next few relevant items rather than dumping entire lists into dashboard cards.

2.3 Reusable design-system components

Create reusable components such as **GlassCard**, **WidgetShell**, **MinimalButton**, **ProgressRing**, **BottomNavigation**, **WidgetHeader**, **EmptyState**, **LoadingState**, **ConfirmActionSheet** and **PermissionStatusCard**. Centralize spacing, radii, colors, typography, elevation, blur strength and motion durations as design tokens.

3. Complete widget inventory — 24 widget types

A. Dashboard and personal OS widgets

Widget	Purpose / behavior	Integration
1. Clock & date	Large clock, date, weekday, next event; optional seconds and alternate clock styles.	Flutter UI + Dart intl/timezone.
2. Daily overview	Today's tasks, completed count, upcoming events and a simple progress indicator.	SQLite/Drift + Riverpod.
3. Quick actions	One-tap Add Task, Add Note, Add Event, Voice Capture and Add Reminder.	Flutter controls + routing + database.
4. Device status	Battery level, charging status and selected device information where supported.	Android platform APIs + Flutter platform channels; Windows adapters where supported.

B. Productivity widgets

Widget	Purpose / behavior	Integration
5. Task list	View, complete and prioritize tasks.	Drift/SQLite + Riverpod.
6. Calendar	Daily, weekly and monthly events.	Flutter calendar UI + date/time utilities.
7. Upcoming deadlines	Exams, assignments, payments and important dates.	Local database + reminder engine.
8. Reminder list	View, snooze, dismiss and edit reminders.	flutter_local_notifications + native Android scheduling.
9. Projects	Progress, subtasks and project status.	SQLite + project module.
10. Notes & quick capture	Ideas, checklists and snippets; optional local attachments.	SQLite + local file storage if needed.

C. Time, routine and focus widgets

Widget	Purpose / behavior	Integration
11. Habit tracker	Daily habits and streaks.	Local habit logs + date utilities.
12. Focus timer	Pomodoro, study sessions and breaks.	Dart timer logic + app lifecycle handling.
13. Routine planner	Morning, college, work and evening routines.	Deterministic workflow engine.
14. Countdown	Exams, birthdays, trips and deadlines.	Date calculations + local database.

D. Media and voice widgets

Widget	Purpose / behavior	Integration
15. Media controls	Play/pause, next/previous and track information.	Native media-session integration; Android MediaSession APIs where appropriate.
16. Voice capture	Speak a task or note instead of typing.	speech_to_text or native speech recognition.
17. Voice playback	Read reminders or selected text aloud.	flutter_tts or native text-to-speech.

Media limitation: the app can control audio it plays itself. Controlling audio from other apps requires relevant Android media-session access and depends on what the other app exposes. Speech recognition may use an online service depending on the device; verify offline recognition if local-only voice input is a strict requirement.

E. Android system integration widgets

Widget / integration	Purpose / behavior	Integration
18. Android home-screen widgets	Show clock, tasks and countdowns outside the app.	Android AppWidget APIs or Jetpack Glance.
19. App shortcuts	Open directly to Add Task, Calendar or Voice Capture.	Android ShortcutManager.
20. Notification actions	Complete, snooze or dismiss reminders from notifications.	NotificationManager + notification action receivers.
21. Share-to-capture	Share text or links into notes/tasks.	Android Sharesheet + intent filters.
22. Quick Settings shortcut	Provide quick entry to a selected action.	Android TileService, subject to platform restrictions.

Platform limitation: ordinary apps cannot freely replace Android's notification shade, lock screen or system-wide interface. Integrate with supported system features and build the app's own polished equivalent for everything else.

F. Customization and personal OS widgets

Widget / module	Purpose / behavior	Integration
23. Screen builder	Create screens, rearrange widgets, resize cards and save layouts.	Custom Flutter widget registry + versioned JSON layout schema.
24. Theme & appearance	Light/dark modes, accent colors, glass intensity, corner radius and typography.	Flutter theme system + locally persisted settings.

Include a Widget Library inside the screen builder so users can add, remove, configure and preview widgets. This is a builder feature, not an extra widget type.

4. Required integrations and technology stack

4.1 Core app foundation

Integration / library	Purpose
Flutter + Dart	Shared UI and application logic for Android and Windows.
Kotlin + Android SDK + Flutter platform channels	Native Android capabilities that are not fully exposed through Flutter packages.
SQLite + Drift	Local relational data, typed queries and reactive database access.
Drift schema migrations	Safe upgrades when the database structure changes.
Riverpod	State management and dependency/provider wiring.
go_router	Navigation, deep links and opening a specific app screen from shortcuts/notifications.
intl + timezone + clock	Date formatting, time-zone-aware scheduling and testable clock logic.
path_provider + file_picker	Local app directories and user-driven file import/export.
flutter_local_notifications	Cross-platform notification integration where supported; native adapters may still be needed.
Android AlarmManager + WorkManager	Time-sensitive reminder scheduling and deferred maintenance, respectively.

4.2 UI/UX building blocks

Integration / library	Purpose
Flutter Material 3 + custom design system	Responsive layout, controls, typography, theme tokens and reusable components.
BackdropFilter / ImageFilter.blur / opacity / gradients	Frosted glass and layered surfaces; built-in Flutter APIs are sufficient for the basic effect.
Flutter drag-and-drop primitives	Reorder widgets; add a grid-layout package only if complex resizing/snapping is needed.
AnimatedSwitcher / Hero / animation controllers	Page transitions, card expansion and feedback animations.
Lucide-style icon set	Consistent icons across modules.
Accessibility semantics + responsive breakpoints	Screen readers, keyboard and touch usability, desktop layouts and reduced-motion support.

4.3 Notifications and scheduling

Integration	Purpose
flutter_local_notifications	Notification creation and actions where supported by the chosen package/version.
Android AlarmManager	Exact or time-sensitive alarm scheduling where appropriate; exact-alarm access may require special user permission.
AndroidX WorkManager	Deferred, deferrable maintenance work—not a substitute for precise alarm timing.
POST_NOTIFICATIONS, NotificationManager, NotificationChannel	Android 13+ notification permission and notification channel management.
BroadcastReceiver + BOOT_COMPLETED + time/date change handling	Reconcile and reschedule eligible reminders after reboot or relevant clock/time-zone changes.
Windows App SDK AppNotificationManager or compatible adapter	Windows toast notifications; confirm packaging and app identity requirements.

4.4 Android system integrations (optional, permission-sensitive)

Integration	Purpose / guardrail
AppWidget / Jetpack Glance	Native Android home-screen widgets.
ShortcutManager	Launcher shortcuts and direct entry points.
Android Sharesheet + intent filters	Receive shared text/URLs from other apps.
NotificationManager + NotificationPolicyAccess	Notification and Do Not Disturb-related actions where user grants access.
UsageStatsManager	Usage information only after the user grants usage access.
NotificationListenerService	Read notifications only if the feature is justified and the user explicitly enables the special access.
AccessibilityService	Only for a clearly defined assistive purpose and compliant use; never use as a general bypass for system restrictions.
TileService	Quick Settings tile, subject to Android rules and user expectations.

4.5 Voice, media and backup

Integration	Purpose
speech_to_text or native speech recognition	Speech-to-text capture; confirm whether the selected engine supports offline recognition on target devices.
flutter_tts or native TextToSpeech	Speak reminders or selected text.
just_audio	In-app audio playback; verify Windows support for the chosen version and add a native adapter if required.
record or native recorder	Optional audio recording and local file handling.
SQLite export/import + optional AES-256 encryption	Local backup portability and confidentiality.
Android Keystore + Windows protected storage	Protect encryption keys using platform-appropriate secure storage.

4.6 Custom screen builder and workflow engine

Build these as internal application modules rather than depending on a remote service. The screen builder uses a registry of approved widget types and a validated, versioned JSON schema. The workflow engine uses a whitelist of deterministic triggers, conditions and actions. Do not allow arbitrary executable code in layout or workflow definitions.

No conversational AI models or cloud services are required. Voice recognition is a platform capability, not a requirement to add a conversational AI model; its network behavior must be checked separately.

5. Application architecture and repository layout

Use a shared Flutter application with platform-specific adapters. Keep domain logic independent from Flutter UI and native APIs so it can be tested without a device.

• **Presentation:** screens, dashboard, reusable widget components, themes, navigation and settings.

• **Application layer:** task/event/reminder use cases, screen-layout operations, voice capture orchestration and workflow execution.

• **Domain layer:** entities, recurrence calculations, reminder occurrence logic, validation rules and deterministic workflows.

• **Data layer:** Drift repositories, migrations, local attachments, settings and backup/export.

• **Platform adapters:** Android alarms, notifications, home widgets, shortcuts, share intents, voice/media and system actions; Windows notifications and desktop-specific behavior.

• **Quality layer:** unit, integration, UI, platform, privacy and performance tests.

Suggested repository structure

```
apps/
mobile/
windows/
packages/
domain/
application/
database/
recurrence/
reminder_engine/
screen_builder/
workflow_engine/
design_system/
platform/
android/
alarms/
notifications/
widgets/
shortcuts/
system_actions/
windows/
notifications/
tests/
unit/
integration/
ui/
docs/
```

6. Local data model

Use SQLite/Drift as the source of truth. Keep UI layout configuration and user preferences local as well. A suggested starting schema is:

Entity / table	Suggested contents
tasks	Title, description, status, priority, due date, project, created/updated timestamps.
projects	Name, description, status, color/icon, dates and progress metadata.
events	Title, start/end, time zone, all-day flag, location/notes and recurrence reference.
reminders	Target entity, scheduled time, state, notification ID, action state and scheduling metadata.
recurrence_rules	Frequency, interval, weekdays/monthly rules, end condition and time zone.
subtasks	Parent task/project, title, status, order and completion time.
notes	Title, content, tags and optional local attachment references.
goals	Name, target, progress method, start/end dates and status.
habits	Name, schedule, target and archived state.
habit_logs	Habit, local date, completion value and optional note.
workflows	Name, enabled state, trigger configuration and version.
workflow_rules	Validated conditions and whitelisted actions.
screens	Screen ID, name, order and layout schema version.
widgets	Widget instance ID, type, screen, position, size and configuration JSON.
settings	Theme, accent, blur level, widget preferences and app options.

Entity / table	Suggested contents
audit_events	Local history of important workflow actions and scheduling failures.

Data design rules

• Use stable IDs and deterministic occurrence IDs for recurring reminder instances.

• Store timestamps in a consistent representation; preserve the relevant time zone for events and recurrence rules.

• Use database migrations and test upgrade paths from every released schema version.

• Validate widget and workflow JSON before saving or executing it; version schemas and provide migration/default behavior.

• Avoid storing more sensitive device information than the feature actually needs.

• Do not add cloud sync. Each device has its own local database; transfer is via explicit export/import unless a future requirement deliberately changes this.

7. Reminder reliability architecture

A reminder must be stored durably before the operating system is asked to schedule it. The database is the source of truth; native scheduling is a derived state that can be rebuilt.

1. User creates or edits a reminder.

2. Validate the requested time, recurrence and target entity.

3. Commit the reminder and its occurrence metadata to SQLite.

4. Calculate eligible upcoming occurrences and assign deterministic occurrence/notification IDs.

5. Schedule using the native adapter: Android AlarmManager for time-sensitive reminders where appropriate; platform notification scheduling for supported Windows behavior.

6. Record the scheduling result and any permission/error state locally.

7. On reboot, time-zone/date change, permission change, app startup or relevant edits, run idempotent reconciliation against the database.

8. Cancel obsolete scheduled occurrences and reschedule changed ones.

9. Provide a permission/status screen and a clear recovery path when notifications or exact-alarm access are disabled.

Scheduling policies

• **AlarmManager:** use for time-sensitive alarms where appropriate. Exact alarms can require special access and are not guaranteed on every device.

• **WorkManager:** use for deferred maintenance, cleanup, reconciliation and export tasks; do not treat it as a precise alarm clock.

• **Android permissions:** handle notification permission on Android 13+ and exact-alarm access where applicable. Explain why access is needed before sending users to system settings.

• **Reboot and clock changes:** handle BOOT_COMPLETED and relevant time/date/time-zone changes where permitted, then reconcile future schedules.

• **Catch-up policy:** define what happens when a reminder's time passed while the app/device was unavailable—e.g., show a bounded missed-reminder summary rather than flooding the user.

• **Rolling window:** schedule a bounded set of future occurrences for recurring items and refresh it as time advances.

• **Idempotency:** repeated reconciliation must not create duplicate notifications.

• **Windows:** verify toast notification requirements, app identity/packaging and cancellation/query limitations for the selected implementation.

• **Honest guarantees:** force-stop behavior, manufacturer battery management, denied permissions and OS restrictions can delay or prevent delivery. Never promise perfect delivery.

Acceptance target (project goal, not a platform guarantee): at least 99% of eligible reminders with required permissions enabled appear within 60 seconds in normal-condition test runs. Measure this on real devices and report exceptions.

8. Screen builder and deterministic workflow engine

8.1 Screen builder requirements

• Create, rename, duplicate, reorder and delete screens.

• Add widgets from a searchable library; preview and configure each one.

• Reorder widgets with drag-and-drop; support sensible size presets and optional resizing.

• Save screen layout, widget order, size, configuration and theme locally.

• Use a versioned JSON schema and a registry of known widget types; unknown or invalid widgets should fail safely and offer recovery.

• Provide templates such as Today, Study, Work, Personal, Focus and Media.

• Support phone and desktop layouts without assuming that every Android home-screen widget can share the same rendering code.

8.2 Workflow engine requirements

• Triggers: time, due date, task state change, routine start/end, habit check-in or user action.

• Conditions: explicit deterministic checks such as priority, project, time range or completion status.

• Actions: create local reminder, show notification, update a permitted field, open a screen or log a result.

• Use an allowlist of supported actions; validate inputs and reject arbitrary code.

• Add enable/disable controls, dry-run/preview where useful, an audit trail and clear error handling.

• Prevent infinite loops and duplicate execution using occurrence IDs, execution limits and idempotency keys.

9. Android and Windows platform limitations

Android

• Notification delivery depends on permission, channel settings, device state and manufacturer battery-management behavior.

• Exact alarms may require special access and should be requested only when the feature qualifies and truly needs precision.

• Ordinary apps cannot freely control all other apps, close arbitrary apps, capture every system-wide gesture, replace the system notification shade or change protected settings.

• UsageStatsManager, NotificationListenerService, DND access and AccessibilityService have separate user-granted access and should be optional, narrowly scoped and clearly explained.

• Native home-screen widgets use Android-specific APIs and have different constraints from full Flutter in-app widgets.

• Offline speech recognition depends on device/OS language packs and recognition service support; test actual target devices.

Windows

• Use Windows-specific notification support and validate packaging/app identity requirements early.

• A Flutter desktop interface should adapt to mouse, keyboard, resizing and larger screens rather than merely stretching the phone layout.

• Verify every plugin's current Windows support before choosing it; `just_audio`, recording and notification packages may need a native adapter or a different implementation.

• Local-only means Android and Windows maintain separate databases unless the user manually exports/imports data. No automatic cross-device sync is included.

10. Staged implementation roadmap

Stage 1 — Foundations

Indicative timing: Weeks 1–2

Implementation: Create repo and app shells; Flutter setup for Android/Windows; design tokens; navigation; Drift schema and migrations; repositories; Riverpod; local settings; app lifecycle and error logging.

Exit criteria: Working shell, database tests, basic navigation and theme switching.

Stage 2 — Core productivity

Indicative timing: Weeks 3–4

Implementation: Tasks, projects, subtasks, notes, events/calendar, recurrence model, daily overview, task list, deadlines and quick actions.

Exit criteria: Users can create/edit/complete tasks and events; dashboard reads real local data.

Stage 3 — Reminder engine

Indicative timing: Weeks 5–6

Implementation: Native Android alarm adapter, notification channels/permission UX, notification actions, occurrence IDs, reconciliation, reboot/time-zone recovery, missed-reminder policy and Windows notification spike.

Exit criteria: Reliable reminder workflow tested on real Android devices and Windows package.

Stage 4 — Customization

Indicative timing: Weeks 7–8

Implementation: Widget registry, versioned layout schema, screen builder, drag-and-drop, size presets, theme editor, widget library and starter templates.

Exit criteria: Users can build and save custom screens without arbitrary code execution.

Stage 5 — Android/Windows integrations

Indicative timing: Weeks 9–10

Implementation: Android home-screen widgets, shortcuts, share-to-capture, Quick Settings tile if justified, voice capture/TTS, media controls and desktop-specific layout/notification adapters.

Exit criteria: Core native integrations work with clear permissions and platform-specific fallbacks.

Stage 6 — Hardening and release

Indicative timing: Weeks 11–12+

Implementation: Backup/export/import, optional encryption, accessibility, performance profiling, manufacturer/device tests, Windows installer/package, privacy review, licensing audit, crash/error handling and release docs.

Exit criteria: A tested MVP/release candidate with documented limitations and recovery steps.

Ship a useful version after Stage 3. Screen builder and advanced integrations can follow without blocking the core productivity app.

11. Testing and acceptance plan

Test area	Required checks
Unit tests	Recurrence calculations; time-zone boundaries; deadline sorting; task state transitions; habit streaks; workflow conditions/actions; stable occurrence IDs; validation of screen JSON.
Database tests	CRUD, transactions, migrations from previous schema versions, backup/restore and invalid-data handling.
Reminder integration tests	Create/edit/cancel; permission denied/granted; exact-alarm access; duplicate prevention; reboot; time-zone/date changes; missed-reminder catch-up; notification actions.
Real Android devices	Test across supported Android versions and multiple manufacturers; Doze/battery saver; app backgrounding; force-stop behavior; notification channel changes; reboot and permission revocation.
Windows tests	Installer/package, app launch, notification display and cancellation, keyboard/mouse navigation, resizing, file import/export and upgrade behavior.
UI/UX tests	Small and large screens, tablet/desktop breakpoints, drag/drop, theme persistence, accessibility semantics, high text scale and reduced motion.
Privacy/security tests	Verify no network dependency for core features; inspect permissions; test local file handling; encryption/key storage if enabled; ensure logs avoid sensitive content.
Performance tests	Startup time, scrolling, database query responsiveness, blur cost, memory usage, animation smoothness and behavior on lower-end devices.

Release acceptance checklist

■ Core tasks, calendar, notes and projects work offline.

■ No account, backend or AI model is required for the core app.

■ Every dashboard widget is driven by the same local data source and updates consistently.

■ Invalid widget/workflow configurations fail safely and do not crash the app.

■ Reminders are stored before scheduling; reconciliation is idempotent; failures are visible to the user.

■ Notification permissions and exact-alarm restrictions are explained clearly.

■ Backup/restore is tested; optional encryption is tested if shipped.

■ Android and Windows layouts are individually usable and polished.

■ Third-party package versions, support status and licenses have been reviewed.

■ Known OS restrictions and delivery limitations are documented.

12. Integration count and prioritization

The 24 entries in the widget catalog are user-facing widget types. Underneath them, the plan uses a smaller number of shared foundational dependencies plus platform APIs and internal modules. The earlier broad integration inventory of roughly 30 areas should be treated as an approximate capability map, not 30 separate third-party packages.

Category	Scope
24 widget types	User-facing modules in the dashboard/widget library.
Core foundation	Flutter/Dart, Kotlin/Android SDK, SQLite/Drift, Riverpod, routing, time utilities, local file access.
Native notification systems	Android AlarmManager, WorkManager, NotificationManager/receivers; Windows notification adapter.
UI design system	Custom Flutter components, blur/animation APIs, drag-and-drop, icon set and accessibility.

Category	Scope
Advanced optional integrations	Home-screen widgets, launcher shortcuts, share intents, Quick Settings tile, media sessions, voice capture/TTS, device usage/notification access, recording and encryption.
Internal modules	Screen builder, widget registry, deterministic workflow engine, recurrence engine, reminder reconciliation, backup/restore.
Required AI/cloud services	0 required.

Recommended MVP priority

• **Must ship first:** design system, dashboard, tasks, calendar/events, notes, local database, basic reminders, notification permissions, local settings and backup/export basics.

• **Next:** projects, routines, habits, focus timer, screen builder, widget library and themes.

• **Platform integrations:** Android home-screen widgets, shortcuts, share-to-capture, voice input/output and Windows notifications.

• **Optional advanced features:** cross-app media controls, Quick Settings tile, UsageStatsManager, NotificationListenerService, DND-related actions, audio recording and encrypted backup. Add only when there is a clear user need and permission explanation.

13. Final implementation recommendation

Build the design system and the first four dashboard widgets before integrating every native Android feature. Once the visual foundation is right, each new widget can reuse the same card, typography, animation and interaction patterns. This is the best way to create a coherent, premium-looking personal OS rather than 24 separate mini-apps competing for attention.

Keep the system local-first, deterministic, permission-transparent and modular. Treat reminders as a reliability-critical subsystem, screen layouts as validated data, and platform features as optional adapters with clear fallbacks.

Version and package compatibility note

Before locking package versions, verify current Android/Windows support, maintenance status, licensing, minimum SDKs and packaging constraints. Test the actual release build—not only debug mode—on real devices and a clean Windows installation.

End of master implementation plan